6.1 Continuous Integration Using GitHub Actions

GitHub Actions was configured to automate the building and testing of the LMS. The workflow runs whenever changes are pushed to the `main` branch or a pull request targets it.

The workflow performs the following steps:

1. Checks out the project repository.
2. Installs the required C++17, SQLite, libsodium, CMake and GoogleTest dependencies.
3. Configures the project using CMake.
4. Builds the LMS application and test executable.
5. Runs the automated tests using CTest.

Result: The GitHub Actions workflow completed successfully. The project configured and built without errors, and all 104 automated tests passed, with zero failures. The test execution took 2.36 seconds.

This provides automated verification that the project builds and its tests pass when changes are submitted to the repository.

6.2 Static Code Analysis Using Cppcheck

Cppcheck version 2.22.0 was used to analyse the C++ source files. The scan covered 32 source files using warning, style, performance and portability checks.

The command used was:

```
cppcheck --enable=warning,style,performance,portability \
--language=c++ --std=c++17 --inline-suppr \
--suppress=missingIncludeSystem \
-I include src include
```

The analysis reported 23 findings: 22 style suggestions and one performance suggestion. No errors or warnings were reported.

The findings included parameter names shadowing existing getter functions, suggestions to use standard library algorithms instead of raw loops, and one suggestion to use a constructor initialisation list.

These findings were reviewed as code-quality recommendations. They were not reported as corrected because no corresponding source-code changes were made during this analysis.

6.3 Cyclomatic Complexity and Code Metrics Using Lizard

Lizard was used to measure the size and complexity of the source code. It analysed 69 files and 356 functions.

Metric	Result
Total non-comment lines of code (NLOC)	4,235
Number of functions	356
Average cyclomatic complexity (CCN)	2.6

Average function length	7.9 NLOC
Functions exceeding CCN 15	2

Two functions exceeded the complexity threshold of 15:

- `toString` in `Enums.cpp`: CCN 18.
- `main` in `main.cpp`: CCN 16.

The average complexity of 2.6 indicates that most functions have relatively simple control flow. The two flagged functions were identified for possible future refactoring.

6.4 Code Quality Checklist

The following areas were reviewed:

- Source code is organised into separate folders.
- User interface, services, domain logic and data access are separated.
- Database operations are handled through the data access layer.
- Input validation is implemented.
- Passwords are stored using salted Argon2id hashes.
- Role-based access restrictions are implemented.
- Database transactions and integrity constraints are used.
- Mark changes are recorded in an audit log.
- Automated tests are available.
- Static analysis and complexity analysis were performed.

6.5 Code Review Findings

The quality review identified the following areas:

Finding	Observation	Action
CR-01	Some parameter names shadow existing getter functions.	Identified for possible renaming.
CR-02	Some loops could use standard algorithms.	Retained for possible improvement.
CR-03	One constructor could use an initialisation list.	Identified for possible improvement.
CR-04	`Enums.cpp` contains a function with CCN 18.	Identified for possible refactoring.
CR-05	`main.cpp` has CCN 16.	Identified for possible refactoring.

The findings were documented for future maintenance. No fixes are claimed where no source-code changes were made.